

ATCHAT

A robust digital mode for amateur radio — pictures, files and a multi-station text net, inside one 2.7 kHz SSB channel.

ATCHAT is a digital mode for amateur radio. It aims to be as easy to run as SSTV, but far more robust, and to do things SSTV cannot: move **images and files with error correction**, and host a **multi-station text net** — all inside one ordinary **2.7 kHz SSB** channel. It takes its cues from KG-STV, EasyPal and HSModem and builds them into a single coordinated on-air protocol.

Where it stands today. ATCHAT is a full software design with a *real* working modem: it genuinely turns data into audio and decodes audio back into data, with real bit errors under real simulated noise. The “channel” is currently a local network link that carries that audio between stations; it is not yet connected to a sound card or an SDR. Everything below is implemented and tested in software.

The one-paragraph pitch

Start the app. Your station finds the net on its own — no “who’s the server?” configuration. You type in a shared channel that everyone sees, or address one station privately. You drop a photo in; it goes out block by block with a checksum on every block, and if the far end misses a few blocks in a fade it asks for **just those** and the picture still arrives pixel-perfect. While that picture is transferring, **the chat keeps working** — ATCHAT deliberately leaves gaps in the file stream so messages and the net heartbeat always get through. If the link gets noisy, the sender **automatically shifts to a slower, tougher modulation**. If a station — even the one coordinating the net — disappears, another one **takes over automatically**, and a station that drops out mid-transfer **resumes where it left off** when it comes back.

Capabilities

Multi-station net, zero configuration

Stations discover each other automatically; the first on frequency starts coordinating. A live **roster** shows who is active, who has gone quiet, and drops stations that stay gone.

Public and private messaging, together

Broadcast chat everyone sees; **directed chat** addressed to one callsign that only that station displays. Text uses the most robust modulation, so short messages punch through conditions that break a file transfer.

Chat keeps flowing during a transfer

ATCHAT builds in a **control window**: it pauses the block stream regularly so chat and the heartbeat always have room. The window **widens automatically** when it detects contention, then narrows again.

Survive a dropout and resume

A station can lose its link and keep all its state. On reconnect it re-announces itself; the other side **re-sends the blocks it still owed**. The transfer finishes the **missing tail only** — never a restart.

Two front ends

A **cross-platform GUI** (Windows / macOS / Linux) with Channel, Stations, NET and Monitor tabs plus real audio output, and a **command-line client** (`/chat` , `/msg` , `/sendimage` , `/sendfile` , `/drop` , `/reconnect`).

Self-healing coordination

One station beacons the roster. If it goes off the air the **next station in line** takes over — and if that one is also gone, the one after it. The order is the same for everyone, so it needs no negotiation, and it does **not** stop at a single designated backup.

Image & file transfer with ARQ

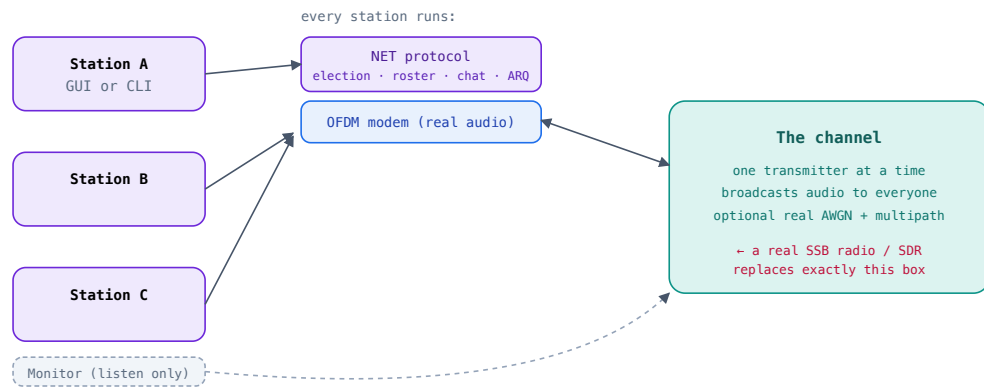
Any file is split into 220-byte blocks, each checksummed. The receiver replies with the **list of blocks it is still missing**; the sender resends **only those**. Delivery is **bit-exact**. Send to **ALL** or to **one callsign**.

Adaptive robustness

Transfers start on the faster **QPSK**. If a recovery round shows the link is losing more than ~15% of what was sent, the sender **drops to BPSK** — slower but far more noise-tolerant — for the rest of that transfer. The receiver needs no warning.

Passive monitoring

A **monitor** listens without ever transmitting and without appearing in the roster. It shows the live **waterfall, spectrum and scope** and a running decode strip ("TA1ABC → ALL | CHAT | decoded", or "undecoded").



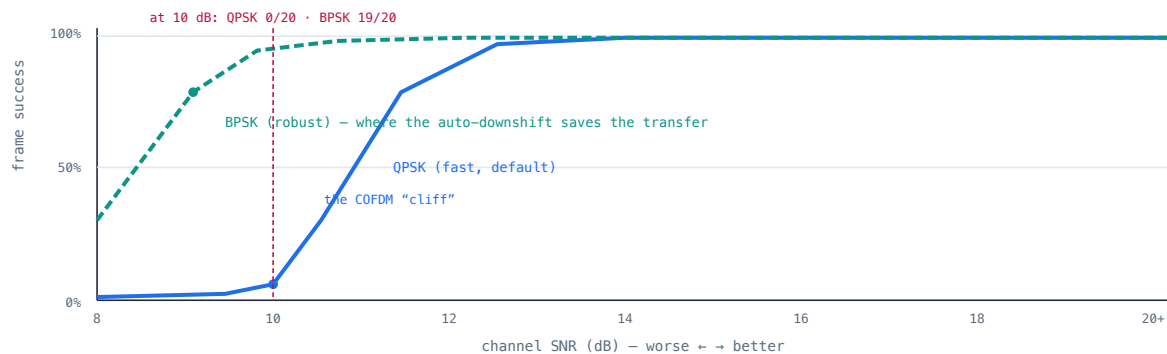
Every station carries all the intelligence. The channel only enforces “one at a time” and passes the audio on — that is the box a real radio takes over.

A session, start to finish

- 1 **Three operators** open ATCHAT and connect to the channel.
- 2 Nobody hears a beacon, so after ~24 s the first station starts beaoning and becomes the net coordinator; the next station is marked as its backup. All three now share a roster.
- 3 **TA1ABC:** `net is open` → everyone sees it. **TA3GHI** replies privately to TA1ABC: `meet on 40 later?` — only TA1ABC sees that line.
- 4 **TA1ABC sends a group photo.** It streams out in blocks; every few blocks the channel opens briefly.
- 5 Mid-transfer, **TA2DEF** types `great pic` — it lands in one of those windows. ATCHAT notices the contention and opens the windows more often for a while.
- 6 **TA3GHI** had a bad few seconds and missed blocks 12, 19, 31. After the sender's "end", TA3GHI asks for exactly those three; they come again and the photo is complete and pixel-identical for all three stations.
- 7 **TA1ABC's laptop sleeps.** The beacon stops. ~24 s later **TA2DEF** (the backup) takes over coordination seamlessly.
- 8 **TA1ABC wakes up**, reconnects, re-announces. It does **not** try to grab coordination back — there is a live beacon. It rejoins as a normal station.

Performance envelope

Measured, not modelled — real modulation through real simulated impairments.



Without FEC, QPSK falls off a cliff around 10–12 dB — exactly where the automatic BPSK downshift, backed by ARQ, keeps the transfer alive.

Situation	Behaviour
Clean channel	100% success, any size tested
Moderate noise (≥ 18 dB SNR)	100%
Marginal (14–16 dB)	small hit; ARQ mops it up
Weak (10–12 dB)	QPSK cliff → automatic BPSK downshift rescues it
Mild multipath (echo inside the 8 ms guard)	rides through
Strong / long multipath	breaks on purpose — no equaliser yet
Raw throughput	≈ 475 B/s at QPSK, ≈ 237 B/s at BPSK, minus the control-window overhead that keeps chat alive

The pieces

Piece	What it does
Modem	Real OFDM: 8 kHz audio, 256-point FFT, 77 subcarriers in ~312–2688 Hz, 8 ms guard. Preamble + header + data symbols, CRC-32 per frame, no FEC.
Channel	Carries the audio, enforces one-transmitter-at-a-time, can add real AWGN and multipath. Holds no protocol logic.
Client (station)	All the intelligence: listen-before-talk, election & failover, roster, chat, file transfer + ARQ, drop/resume.
Monitor	Listens only. Scope + spectrum + waterfall + decode strip.

A reference implementation in **Python** and a full **Rust** port track each other bit-for-bit.

What's next

- **Forward error correction** (LDPC/RS) so weak-signal frames self-heal instead of leaning entirely on ARQ.
- **Channel estimation / equaliser** so real-world multipath beyond the guard interval stops being a hard wall — this also unlocks true per-subcarrier adaptive bit loading.
- **Sound-card audio** so two radios can pass ATCHAT over a real SSB link on the bench.
- **SDR / RF integration.** By design this should touch only the channel layer; the modem and the protocol stay as they are.

Getting it

Prebuilt GUI and channel binaries for **Windows (MSI + zip)**, **macOS** (Intel + Apple Silicon) and **Linux** are published on the [GitHub Releases page](#). To run from source you only need `numpy` for the Python side, or a Rust toolchain for the `rust/` workspace. See `README.md` for the exact commands and **PROTOCOL.md / the Protocol Specification** for the full technical detail.